

C3M4_Peer_Reviewed

March 30, 2025

```
[1]: library(ggplot2)
library(mgcv)
```

Registered S3 methods overwritten by 'ggplot2':

method	from
[.quosures	rlang
c.quosures	rlang
print.quosures	rlang

Loading required package: nlme

This is mgcv 1.8-28. For overview type 'help("mgcv-package")'.

1 C3M4 Peer Reviewed Assignment

1.0.1 Outline:

The objectives for this assignment:

1. Observe the difference between GAMs and other regression models on simulated data.
2. Review how to plot and interpret the coefficient linearity for GAM models.

General tips:

1. Read the questions carefully to understand what is being asked.
2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

```
[1]: # Load packages
library(ggplot2)
library(mgcv)
```

Loading required package: nlme

This is mgcv 1.8-31. For overview type 'help("mgcv-package")'.

2 Problem 1: GAMs with Simulated Data

In this example, we show how to check the validity of a generalized additive model (GAM) (using the `gam()` function) using simulated data. This allows us to try and understand the intricacies of `gam()` without having to worry about the context of the data.

2.0.1 1. (a) Simulate the Data

Let $n = 200$. First, construct three predictor variables. The goal here is to construct a GAM with different types of predictor terms (e.g., factors, continuous variables, some that will enter linearly/parametrically, some that enter transformed).

1. `x1`: A continuous predictor that, we will suppose has a nonlinear relationship with the response.
2. `x2`: A categorical variable with three levels: `s`, `m`, and `t`.
3. `x3`: A categorical variable with two levels: `TRUE` and `FALSE`.

Then, make the response some nonlinear/nonparametric function of

$$\mathbf{x}$$

. For our case, use:

$$\log(\mu_i) = \beta_1 + \sin(0.5x_{i,1}^2) - x_{i,2} + x_{i,3}$$

This model is a Poisson GAM. In a realworld situation, we wouldn't know this functional relationship and would estimate it. Other terms are modeled parametrically. The response has normal noise.

Note that:

1. The construction of

$$\boldsymbol{\mu} = (\mu_1, \dots, \mu_n)^T$$

has the linear predictor exponentiated, because of the nature of the link function.

2. We use $\boldsymbol{\mu}$ to construct

$$\mathbf{y} = (y_1, \dots, y_n)^T$$

. The assumption for Poisson regression is that the random variable Y_i that generates y_i is Poisson with mean μ_i .

3. `as.integer(as.factor(VARIABLE))` converts the labels of `VARIABLE` to 1, 2, 3,.. so that we can construct the relationship for these factors.

Plot the relationship of $\mathbf{y}$ to each of the predictors. **Then, split the data into a training (`train_sim`) and test (`test_sim`) set.**

```
[62]: set.seed(0112358)
      # n = number of data points
      n = 200

      x1 <- runif(n, min = 0, max = 9)
      x2 <- factor(sample(c("s", "m", "t"), n, replace = TRUE))
      x3 <- factor(sample(c(TRUE, FALSE), n, replace = TRUE))
```

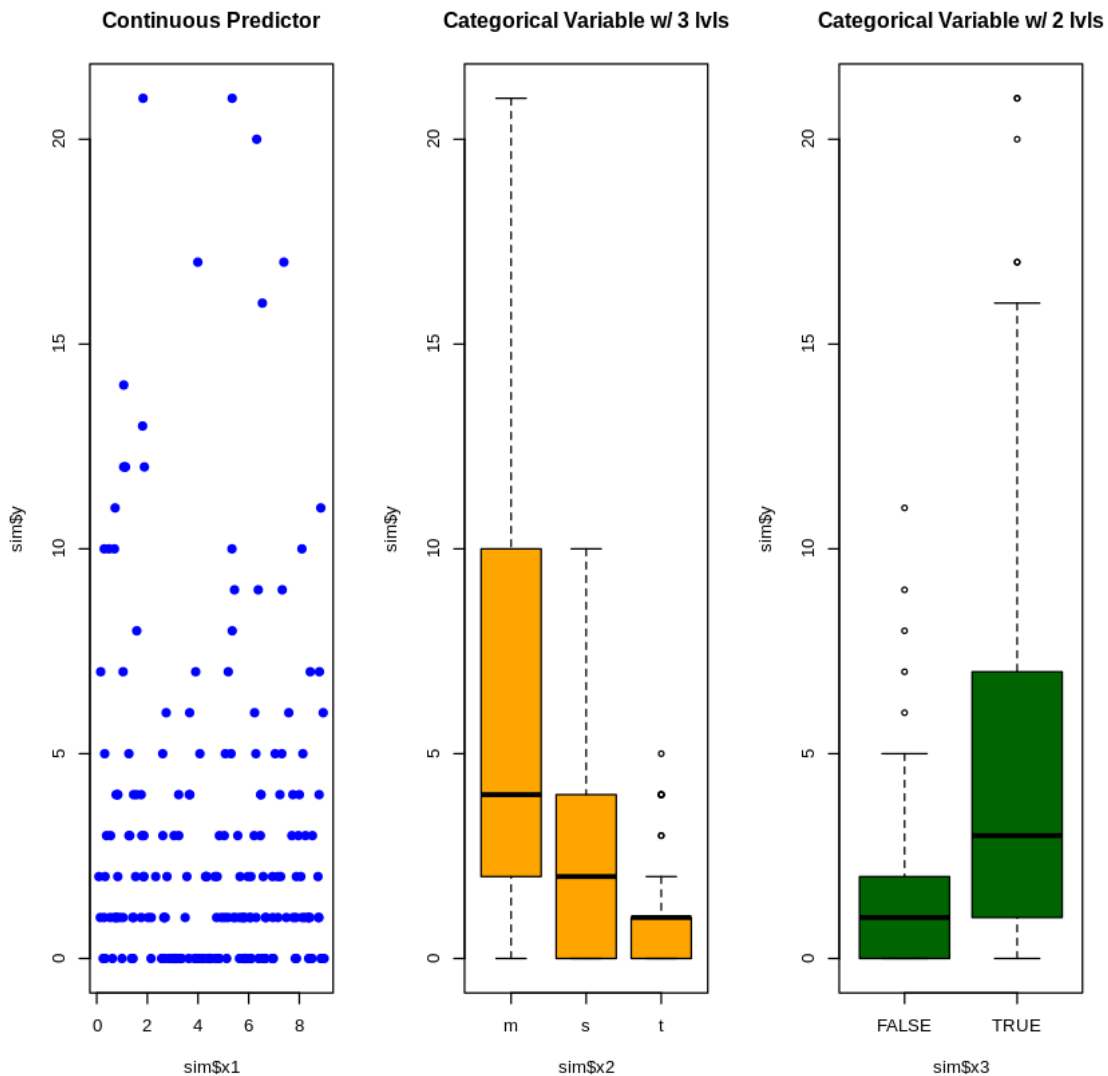
```

mu <- exp(1 + sin(0.5 * x1^2) - as.integer(x2) + as.integer(x3))
y <- rpois(n, lambda = mu)

sim <- data.frame(x1, x2, x3, y)
par(mfrow = c(1, 3))
plot(sim$x1, sim$y, main = "Continuous Predictor", col = "blue", pch = 19)
boxplot(sim$y ~ sim$x2, main = "Categorical Variable w/ 3 lvls", col = "orange")
boxplot(sim$y ~ sim$x3, main = "Categorical Variable w/ 2 lvls", col = "darkgreen")

train_sim <- sim[1:floor(0.8 * n), ]
test_sim <- sim[(floor(0.8 * n) + 1):n, ]

```



2.0.2 1. (b) Other Regression Models

Before jumping straight into GAMs, let's test if other regression models work. What about a regular linear regression model with ordinary least squares, and a generalized linear model for Poisson regression?

First fit a linear regression model to your `train_sim` data. We know that all of the predictors were used to make the response, but are they all significant in the linear regression model? Explain why this may be.

Then fit a Generalized Linear Model (GLM) to the `train_sim` data. Plot three diagnostic plots for your GLM:

1. Residual vs. log(Fitted Values)
2. QQPLOT of the Residuals
3. Actual Values vs. Fitted Values

Using these plots, determine whether this model is a good fit for the data. Make sure to explain your conclusions and reasoning.

```
[53]: # Fit a LM model to the data
lm_model <- lm(y ~ x1 + x2 + x3, data = train_sim)
summary(lm_model)

# Fit a GLM model to the data
glm_model <- glm(y ~ x1 + x2 + x3, family = poisson(link = "log"), data =
  ↪train_sim)

#residual plot
par(mfrow = c(1, 3))

# Create the three specified plots
plot(log(fitted(glm_model)), residuals(glm_model), main = "Residual vs. log
  ↪(Fitted Values)",
      col = "blue", pch = 19)

## qqplot
qqnorm(residuals(glm_model), main = "QQ Plot of the Residuals", col = "orange")
qqline(residuals(glm_model))

#fitted vs actual
plot(test_sim$y, predict(glm_model, newdata = test_sim, type = "response"),
      main = "Actual Values vs. Fitted Values", col = "darkgreen", pch = 19)
abline(0, 1, col = "black")
```

Call:

```
lm(formula = y ~ x1 + x2 + x3, data = train_sim)
```

Residuals:

Min	1Q	Median	3Q	Max
-----	----	--------	----	-----

-7.2029 -2.2406 -0.3326 1.4195 13.8096

Coefficients:

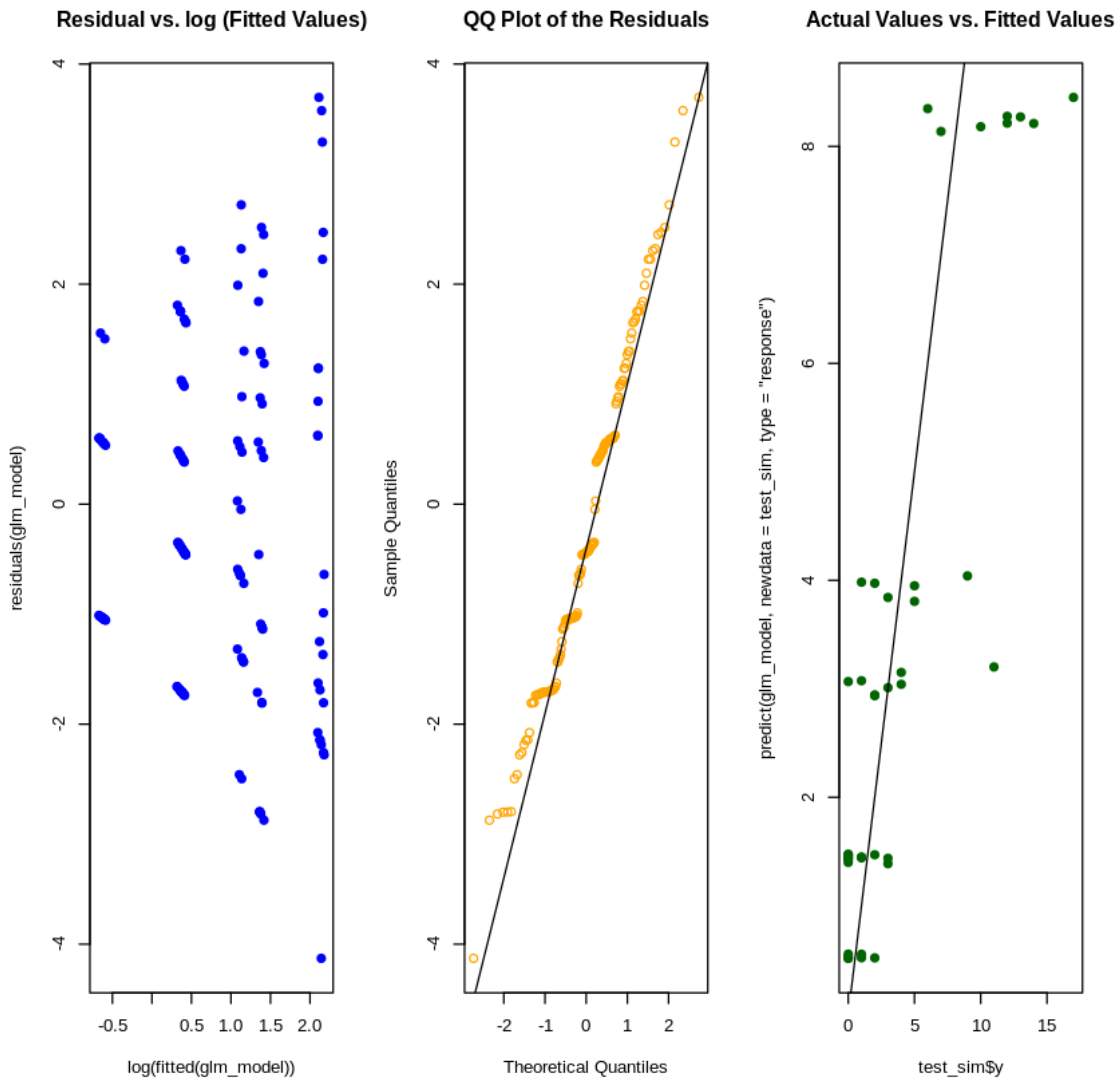
	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	4.443958	0.697579	6.371	2.04e-09	***
x1	0.004121	0.098167	0.042	0.967	
x2s	-3.120411	0.661310	-4.719	5.27e-06	***
x2t	-4.866690	0.630195	-7.723	1.32e-12	***
x3TRUE	2.738941	0.513506	5.334	3.35e-07	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.245 on 155 degrees of freedom

Multiple R-squared: 0.3638, Adjusted R-squared: 0.3474

F-statistic: 22.16 on 4 and 155 DF, p-value: 1.753e-14



1. From the summary of the LM model, we see that only the predictor x_1 is insignificant. This might be because there is no meaningful relationship with y or due to multicollinearity.
2. From the 1st plot, we can observe patterns. On the 2nd plot, we see some deviations from the diagonal line especially on the lower tail. On the 3rd plot, we see that the 45-degree line does not predict the given data well. Thus, the GLM model is a bad fit for the data.

2.0.3 1. (c) Looking for those GAMs

Now, it's time to see how a generalized additive model (GAM) performs! Fit a GAM to the data. Construct the same three plots for your GAM model. Do these plots look better than those of the GLM?

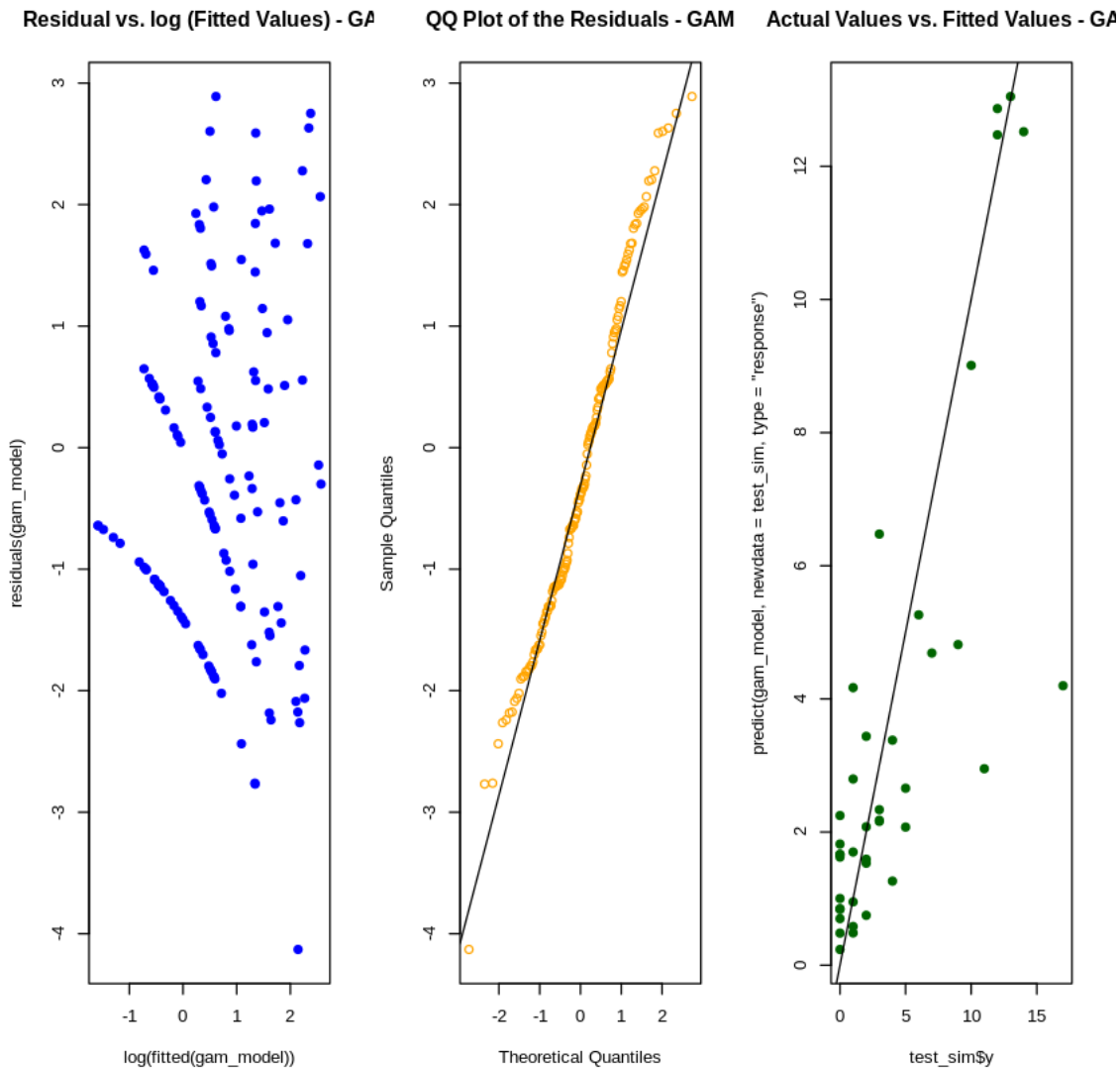
```
[32]: # Fit a GAM model to the data
gam_model <- gam(y ~ s(x1) + x2 + x3, family = poisson(link = "log"), data =
  ↪train_sim)

# Construct the three specified plots
par(mfrow = c(1, 3))

#residual vs fitted
plot(log(fitted(gam_model)), residuals(gam_model), main = "Residual vs. log
  ↪(Fitted Values) - GAM",
      col = "blue", pch = 19)

## qqplot
qqnorm(residuals(gam_model), main = "QQ Plot of the Residuals - GAM", col =
  ↪"orange")
qqline(residuals(gam_model))

#fitted vs actual
plot(test_sim$y, predict(gam_model, newdata = test_sim, type = "response"),
      main = "Actual Values vs. Fitted Values - GAM", col = "darkgreen", pch =
  ↪19)
abline(0, 1, col = "black")
```



From the 1st plot, we can still observe some patterns but the data is now more randomly scattered. On the 2nd plot, we see that the points now follow a straight diagonal line more closely. On the 3rd plot, we see that the 45-degree line does predict the given data. Thus, the GAM model is a good fit for the data.

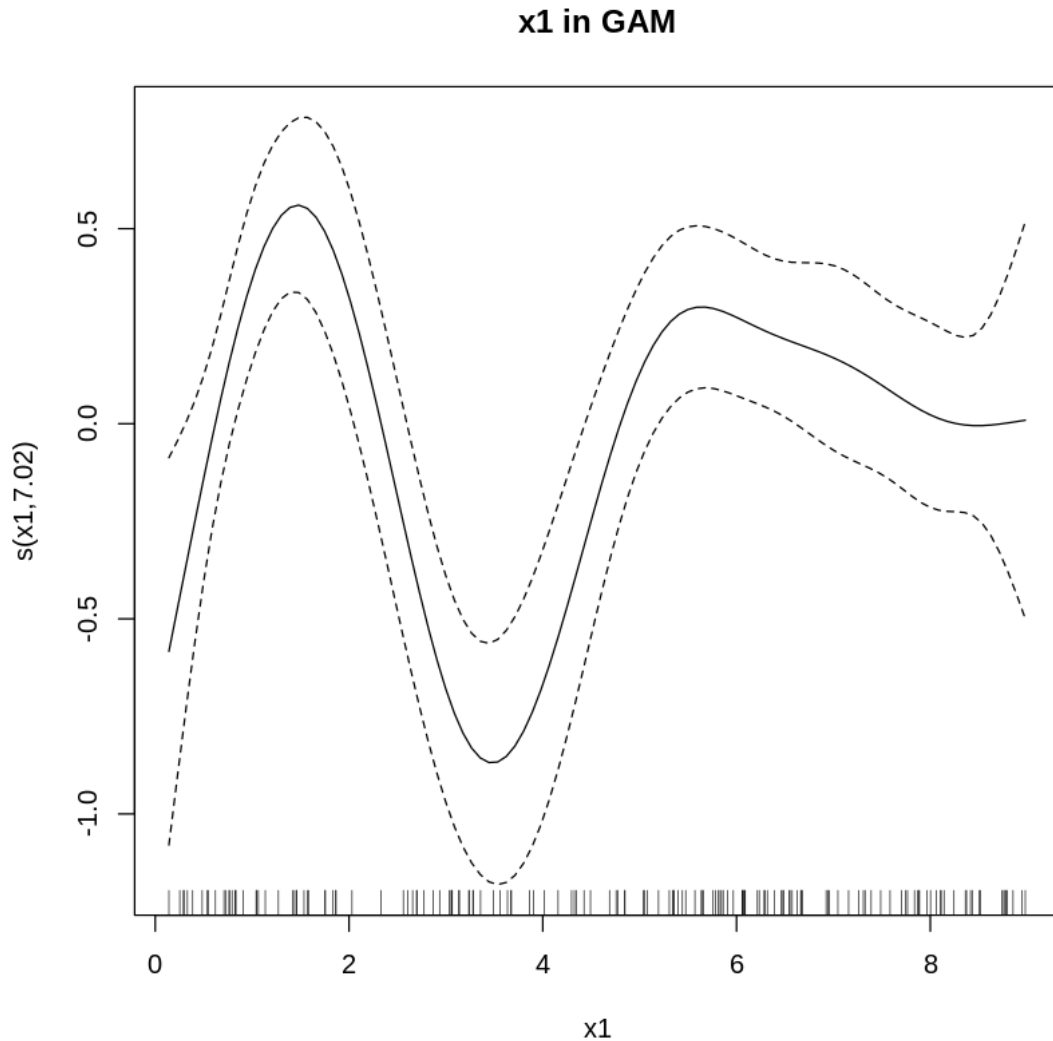
2.0.4 1. (d) Interpreting GAMs

We made a GAM model! However GAMs are harder to interpret than regular linear regression models. How do we determine if a GAM model was necessary? Or, in other words, how do we determine if our predictors have a linear relationship with the response?

Use the `plot.gam()` function in the `mgcv` library to plot the relationship between y and x_1 . Recall that x_1 entered our model as $\sin(0.5x_{i,1}^2)$, and we plotted that relationship in 1.(a). Does your plot

confirm this relationship?

```
[52]: plot.gam(gam_model, select = 1, main = "x1 in GAM")
```



Yes, the plot confirms that x_1 was required and a GAM model was necessary (the plot looks like a sine function and stays within the dotted lines).

2.0.5 1.(e) Model comparison

Compute the mean squared prediction error (MSPE) for each of the three models above (regression model, GLM, and GAM). State which model performs best according to this metric.

Remember, the MSPE is given by

$$MSPE = \frac{1}{k} \sum_{i=1}^k (y_i^* - \hat{y}_i^*)^2$$

where y_i^* are the observed response values in the test set and $\hat{y}_i^*$ are the predicted values for the test set (using the model fit on the training set).

```
[47]: mspe <- function(actual, predicted) mean((actual - predicted)^2)
      printf <- function(...) cat(sprintf(...))

[51]: #mspe for lm
      mspe_lm <- mspe(test_sim$y, predict(lm_model, newdata = test_sim))
      printf("MSPE for LM: %s \n", mspe_lm)

      #mspe for glm
      mspe_glm <- mspe(test_sim$y, predict(glm_model, newdata = test_sim, type = "response"))
      printf("MSPE for GLM: %s \n", mspe_glm)

      # mspe for gam
      mspe_gam <- mspe(test_sim$y, predict(gam_model, newdata = test_sim, type = "response"))
      printf("MSPE for GAM: %s \n", mspe_gam)
```

```
MSPE for LM: 9.80199441647645
MSPE for GLM: 7.69429257950574
MSPE for GAM: 8.25002129779613
```

The GLM model has the lowest MSPE, therefore it performs the best.

3 Problem 2 Additive models with the advertising data

The following dataset contains measurements related to the impact of three advertising medias on sales of a product, P . The variables are:

- **youtube**: the advertising budget allocated to YouTube. Measured in thousands of dollars;
- **facebook**: the advertising budget allocated to Facebook. Measured in thousands of dollars; and
- **newspaper**: the advertising budget allocated to a local newspaper. Measured in thousands of dollars.
- **sales**: the value in the i^{th} row of the sales column is a measurement of the sales (in thousands of units) for product P for company i .

The advertising data treat “a company selling product P ” as the statistical unit, and “all companies selling product P ” as the population. We assume that the $n = 200$ companies in the dataset were chosen at random from the population (a strong assumption!).

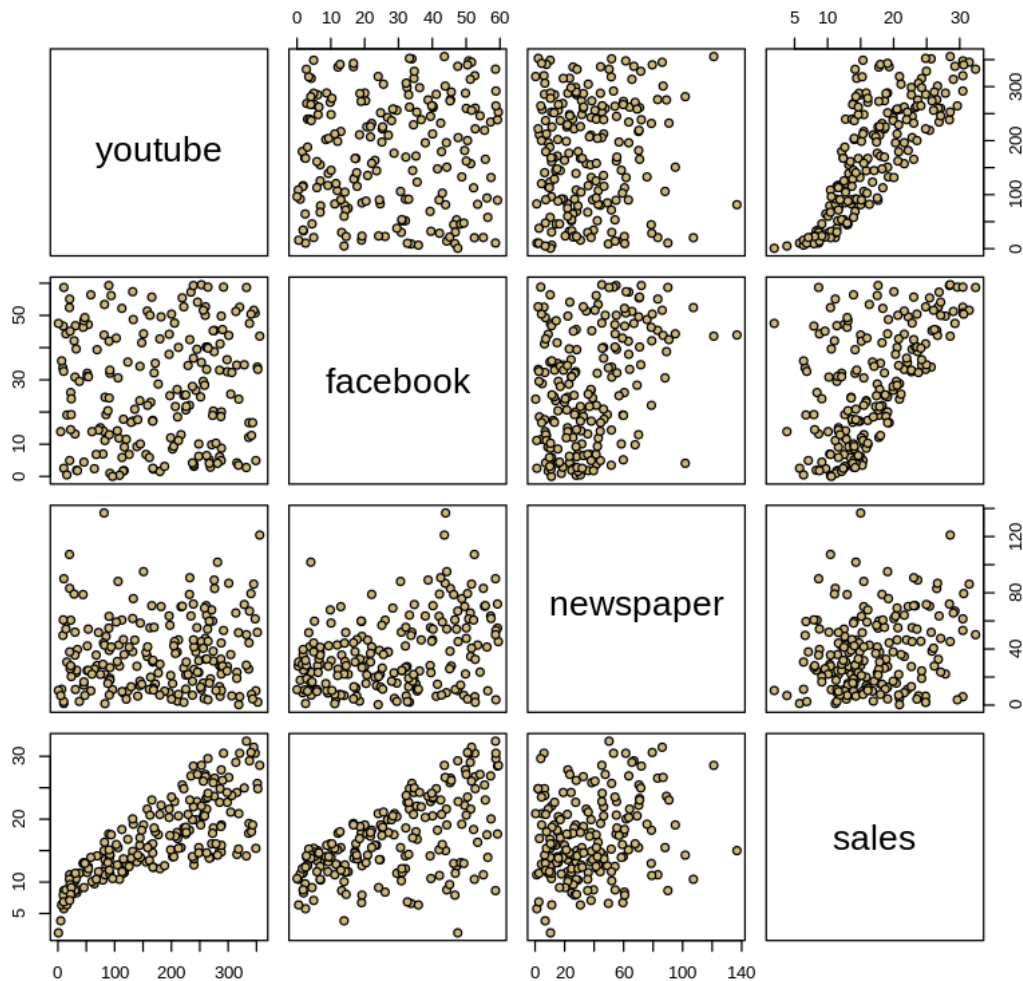
First, we load the data, plot it, and split it into a training set (`train_marketing`) and a test set (`test_marketing`).

```
[54]: # Load in the data
marketing = read.csv("marketing.txt", sep = "")
head(marketing)
pairs(marketing, main = "Marketing Data", pch = 21,
      bg = c("#CFB87C"))
```

A data.frame: 6 × 4

	youtube <dbl>	facebook <dbl>	newspaper <dbl>	sales <dbl>
1	276.12	45.36	83.04	26.52
2	53.40	47.16	54.12	12.48
3	20.64	55.08	83.16	11.16
4	181.80	49.56	70.20	22.20
5	216.96	12.96	70.08	15.48
6	10.44	58.68	90.00	8.64

Marketing Data



```
[21]: set.seed(177) #set the random number generator seed.
n = floor(0.8 * nrow(marketing)) #find the number corresponding to 80% of the
  ↳ data
index = sample(seq_len(nrow(marketing)), size = n) #randomly sample indices to
  ↳ be included in the training set

train_marketing = marketing[index, ] #set the training set to be the randomly
  ↳ sampled rows of the dataframe
test_marketing = marketing[-index, ] #set the testing set to be the remaining
  ↳ rows
dim(test_marketing) #check the dimensions
dim(train_marketing) #check the dimensions
```

1. 40 2. 4

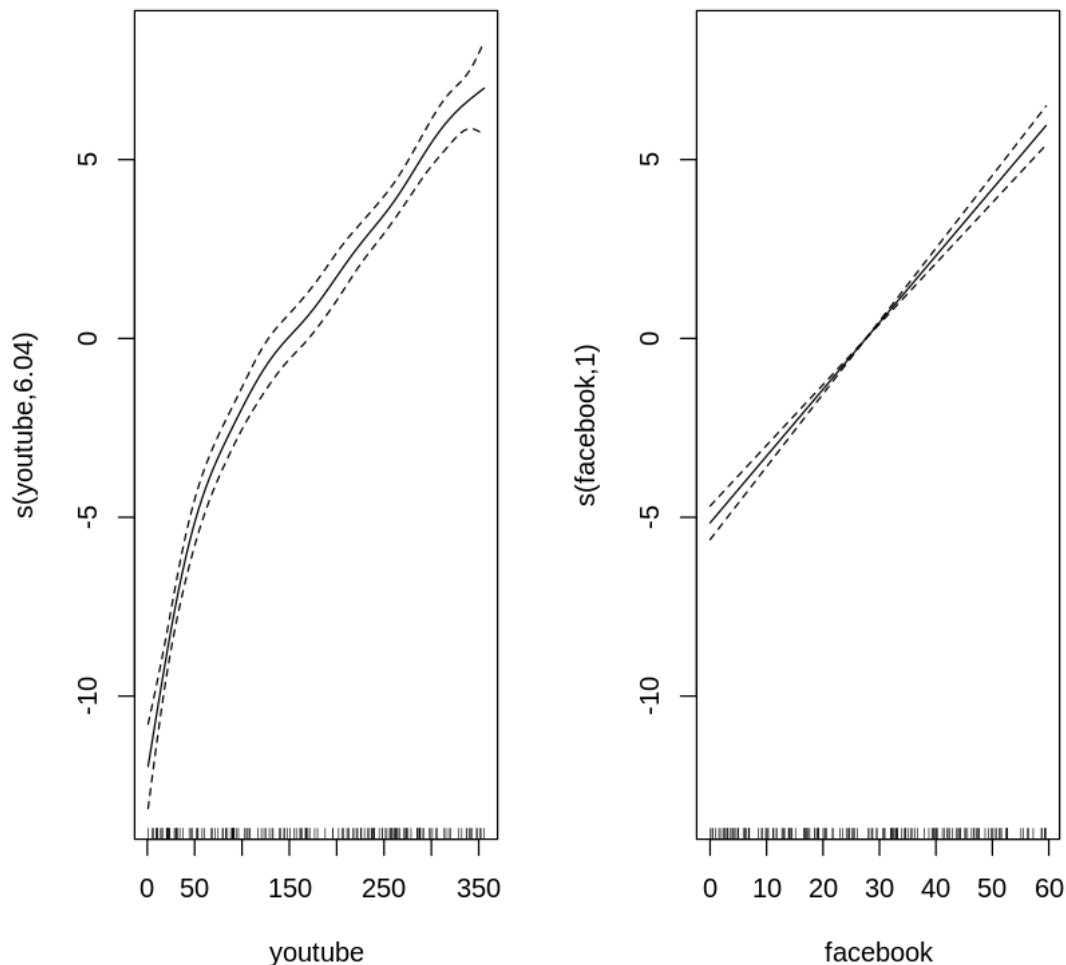
1. 160 2. 4

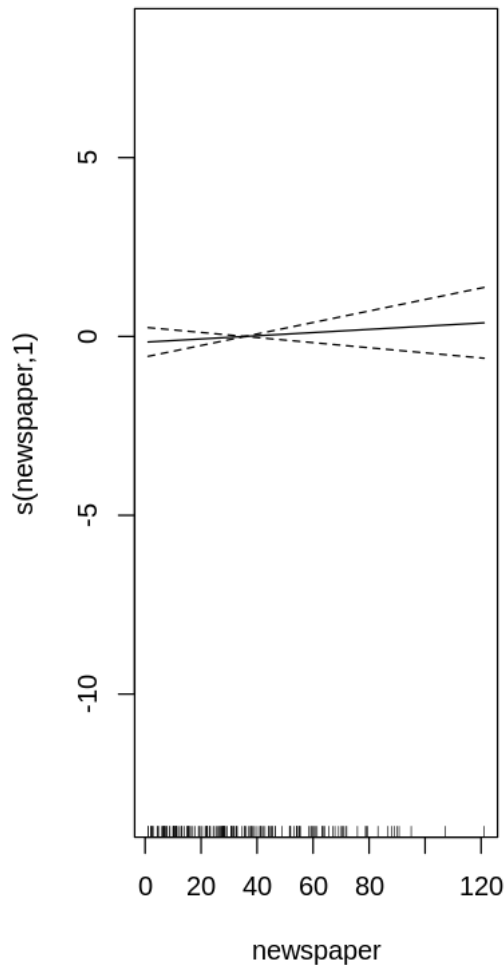
2.(a) Let's try a GAM on the marketing data!

Note that the relationship between `sales` and `youtube` is nonlinear. This was a problem for us back in the first course in this specialization, when we modeled the data as if it were linear. In the last module, we focused on modeling the relationship between `sales` and `youtube`, omitting the other variables. Now it's time to include the additional predictors.

Using the `train_marketing` fit an additive model to the data and store it in `gam_marketing`. Produce the relevant added variable plots using `plot(gam_marketing)`. Comment on the fit of the model.

```
[56]: gam_marketing <- gam(sales ~ s(youtube) + s(facebook) + s(newspaper), data = train_marketing)
      plot(gam_marketing, pages = 2)
```





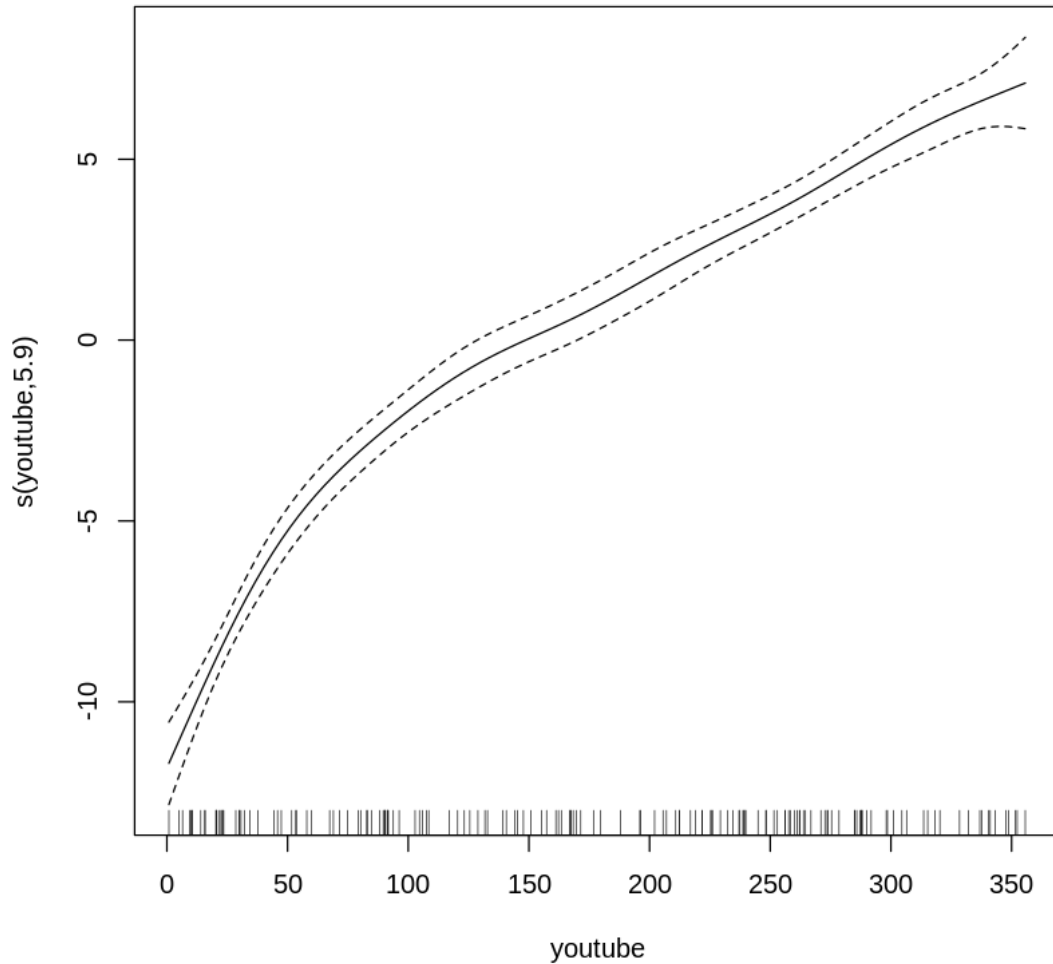
The plots confirm the relationship between sales and youtube is nonlinear, and the other two are not. They also confirm that a GAM model was necessary since all plots stay within the dotted lines. Moreover, we can conclude that newspaper is a statistically insignificant predictor.

2.(b) Semiparametric modeling of the marketing data

Refit the additive model based on your results from 2.(a). That is, if any predictors above should enter linearly, refit the model to reflect that. If any predictors are statistically insignificant, remove them from the model. Store your final model in `semiparametric_marketing`.

```
[60]: semiparametric_marketing <- gam(sales ~ s(youtube) + facebook, data = train_marketing)
```

```
plot(semiparametric_marketing, pages = 2)
```



A new model was build using the results from 2a.

2.(c) Model comparisons

Now, let's do some model comparisons on the test data. Compute the mean squared prediction error (MSPE) on the `test_marketing` data for the following three models:

- `gam_marketing` from 2.(a)
- `semiparametric_marketing` from 2.(b)
- `lm_marketing`, a linear regression model with `sales` is the response and `youtube` and `facebook` are predictors (fit on the `train_marketing` data).

State which model performs best according to this metric.

```
[61]: #mspe for gam
mspe_gam_marketing <- mspe(test_marketing$sales, predict(gam_marketing, newdata =
  test_marketing))
printf("MSPE for GAM Marketing: %s \n", mspe_gam_marketing)

#mspe for semiparametric
mspe_semiparametric_marketing <- mspe(test_marketing$sales,
  predict(semiparametric_marketing,
                                                newdata =
  test_marketing))
printf("MSPE for Semiparametric GAM Marketing: %s \n",
  mspe_semiparametric_marketing)

# mspe for lm
lm_marketing <- lm(sales ~ youtube + facebook, data = train_marketing)
mspe_lm_marketing <- mspe(test_marketing$sales, predict(lm_marketing, newdata =
  test_marketing))
printf("MSPE for LM Marketing: %s \n", mspe_lm_marketing)
```

MSPE for GAM Marketing: 3.43820224517997

MSPE for Semiparametric GAM Marketing: 3.40994916862721

MSPE for LM Marketing: 4.19770116857387

Our adjusted semiparametric model has the lowest MSPE, and performs the best as expected.